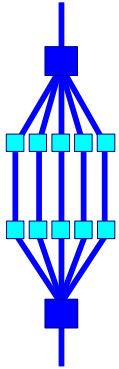
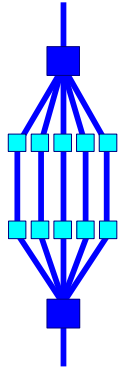


Terminology



- ❑ *OpenMP Team := Master + Workers*
- ❑ *A Parallel Region is a block of code executed by all threads simultaneously*
 - ☞ *The master thread always has thread ID 0*
 - ☞ *Thread adjustment (if enabled) is only done before entering a parallel region*
 - ☞ *Parallel regions can be nested, but support for this is implementation dependent*
 - ☞ *An "if" clause can be used to guard the parallel region; in case the condition evaluates to "false", the code is executed serially*
- ❑ *A work-sharing construct divides the execution of the enclosed code region among the members of the team; in other words: they split the work*

Components of OpenMP



Directives

- ◆ *Parallel regions*
- ◆ *Work sharing*
- ◆ *Synchronization*
- ◆ *Data scope attributes*
 - ☞ *private*
 - ☞ *firstprivate*
 - ☞ *lastprivate*
 - ☞ *shared*
 - ☞ *reduction*
- ◆ *Orphaning*

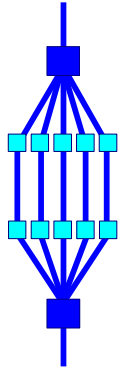
Environment variables

- ◆ *Number of threads*
- ◆ *Scheduling type*
- ◆ *Dynamic thread adjustment*
- ◆ *Nested parallelism*

Runtime environment

- ◆ *Number of threads*
- ◆ *Thread ID*
- ◆ *Dynamic thread adjustment*
- ◆ *Nested parallelism*
- ◆ *Timers*
- ◆ *API for locking*

A more elaborate example



```
#pragma omp parallel if (n>limit) default(none) \
    shared(n,a,b,c,x,y,z) private(f,i,scale)
{
```

```
    f = 1.0;
```

```
#pragma omp for nowait
```

```
    for (i=0; i<n; i++)
        z[i] = x[i] + y[i];
```

```
#pragma omp for nowait
```

```
    for (i=0; i<n; i++)
        a[i] = b[i] + c[i];
```

```
#pragma omp barrier
```

```
        ....
    scale = sum(a,0,n) + sum(z,0,n) + f;
        ....
```

```
} /*-- End of parallel region --*/
```

Statement is executed
by all threads

parallel loop
(work will be distributed)

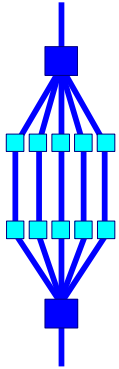
parallel loop
(work will be distributed)

synchronization

Statement is executed
by all threads

parallel region

The parallel region

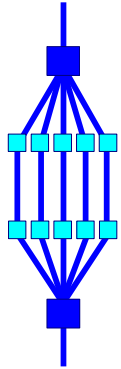


A parallel region is a block of code executed by multiple threads simultaneously

```
#pragma omp parallel [clause[,] clause] ...]
{
    "this will be executed in parallel"
} (implied barrier)
```

```
!$omp parallel [clause[,] clause] ...]
    "this will be executed in parallel"
!$omp end parallel (implied barrier)
```

The parallel region - clauses

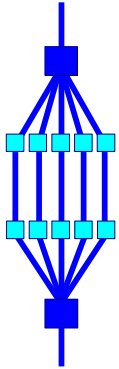


A parallel region supports the following clauses:

if	(<i>scalar expression</i>)	
private	(<i>list</i>)	
shared	(<i>list</i>)	
default	(<i>none shared</i>)	(C/C++)
default	(<i>none shared private</i>)	(Fortran)
reduction	(<i>operator: list</i>)	
copyin	(<i>list</i>)	
firstprivate	(<i>list</i>)	
num_threads	(<i>scalar_int_expr</i>)	

Work-sharing constructs

The OpenMP work-sharing constructs



```
#pragma omp for
{
    . . .
}
```

```
!$OMP DO
```

```
. . .
```

```
!$OMP END DO
```

```
#pragma omp sections
{
    . . .
}
```

```
!$OMP SECTIONS
```

```
. . .
```

```
!$OMP END SECTIONS
```

```
#pragma omp single
{
    . . .
}
```

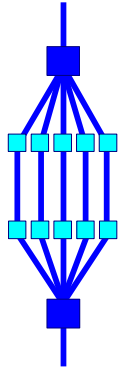
```
!$OMP SINGLE
```

```
. . .
```

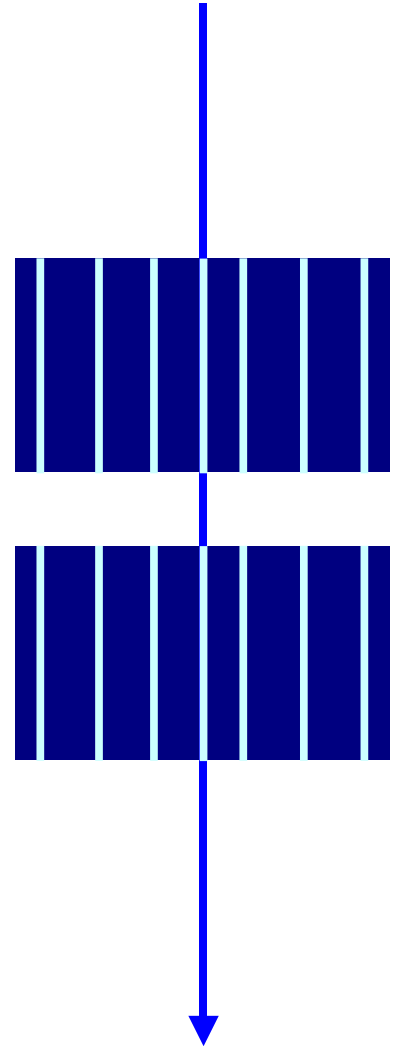
```
!$OMP END SINGLE
```

- ☞ *The work is distributed over the threads*
- ☞ *Must be enclosed in a parallel region*
- ☞ *Must be encountered by all threads in the team, or none at all*
- ☞ *No implied barrier on entry; implied barrier on exit (unless nowait is specified)*
- ☞ *A work-sharing construct does not launch any new threads*

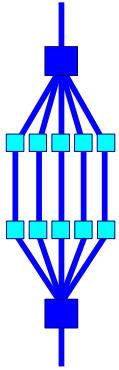
The omp for directive - example



```
#pragma omp parallel default(none) \  
    shared(n,a,b,c,d) private(i)  
{  
    #pragma omp for nowait  
    for (i=0; i<n-1; i++)  
        b[i] = (a[i] + a[i+1])/2;  
  
    #pragma omp for nowait  
    for (i=0; i<n; i++)  
        d[i] = 1.0/c[i];  
  
} /*-- End of parallel region --*/  
    (implied barrier)
```

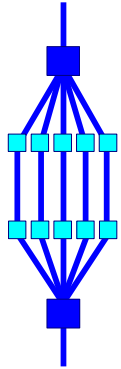


Load balancing



- ❑ *Load balancing is an important aspect of performance*
- ❑ *For regular operations (e.g. a vector addition), load balancing is not an issue*
- ❑ *For less regular workloads, care needs to be taken in distributing the work over the threads*
- ❑ *Examples of irregular workloads:*
 - *Transposing a matrix*
 - *Multiplication of triangular matrices*
 - *Parallel searches in a linked list*
- ❑ *For these irregular situations, the **schedule** clause supports various iteration scheduling algorithms*

The schedule clause/1



**schedule (static | dynamic | guided [, chunk])
schedule (runtime)**

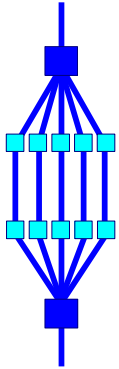
static [, chunk]

- ✓ *Distribute iterations in blocks of size "chunk" over the threads in a round-robin fashion*
- ✓ *In absence of "chunk", each thread executes approx. N/P chunks for a loop of length N and P threads*

Example: Loop of length 16, 4 threads:

TID	0	1	2	3
no chunk	1-4	5-8	9-12	13-16
chunk = 2	1-2 9-10	3-4 11-12	5-6 13-14	7-8 15-16

The schedule clause/2



dynamic [, chunk]

- ✓ *Fixed portions of work; size is controlled by the value of chunk*
- ✓ *When a thread finishes, it starts on the next portion of work*

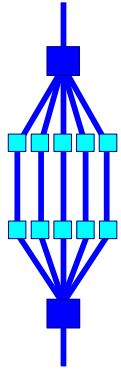
guided [, chunk]

- ✓ *Same dynamic behaviour as "dynamic", but size of the portion of work decreases exponentially*

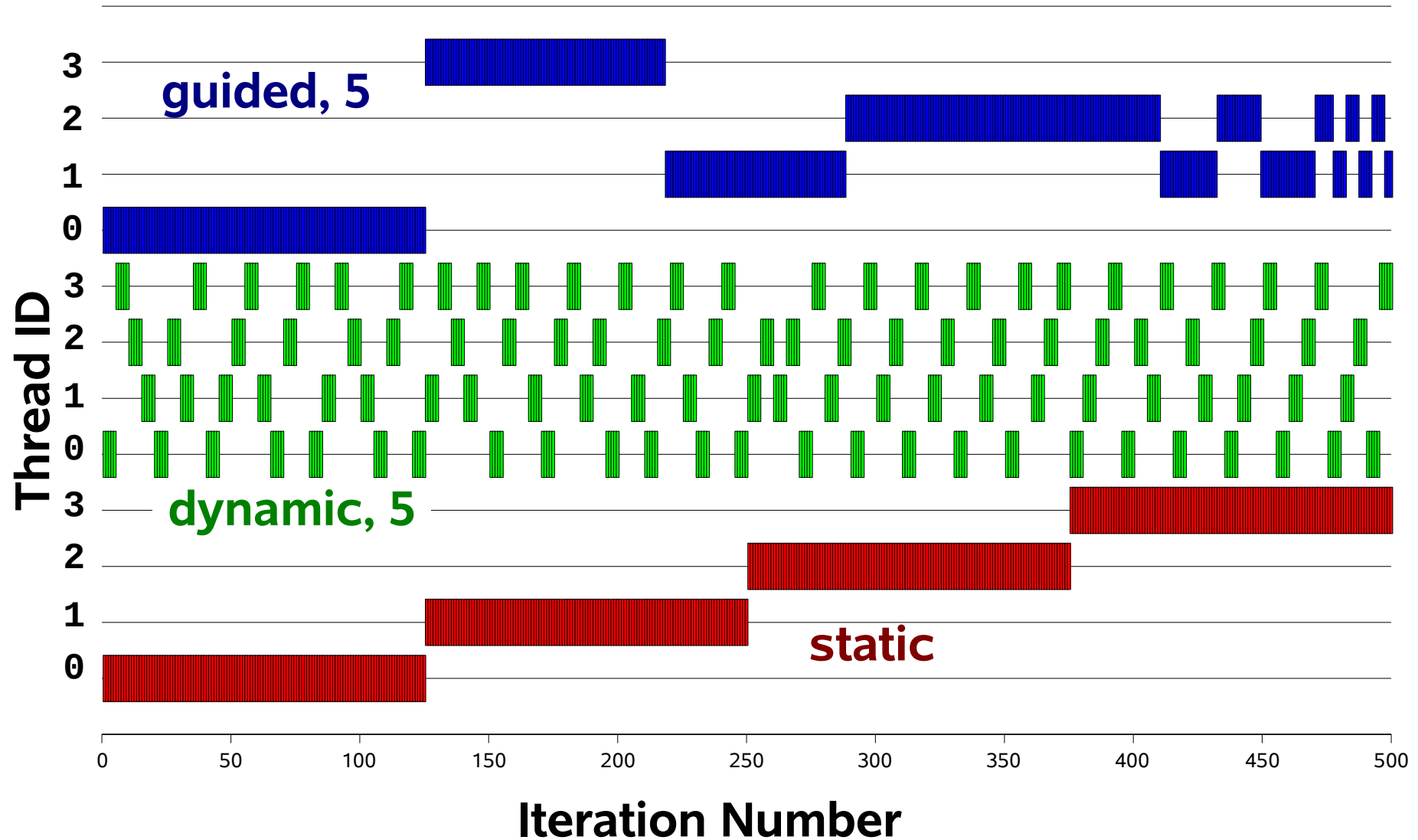
runtime

- ✓ *Iteration scheduling scheme is set at runtime through environment variable **OMP_SCHEDULE***

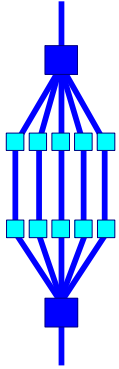
The experiment



500 iterations on 4 threads



The SECTIONS directive



The individual code blocks are distributed over the threads

```
#pragma omp sections [clause(s)]  
{  
  #pragma omp section  
    <code block1>  
  #pragma omp section  
    <code block2>  
  #pragma omp section  
    :  
}
```

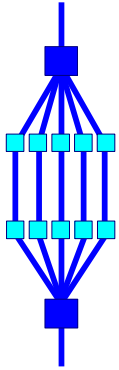
```
!$omp sections [clause(s)]  
!$omp section  
    <code block1>  
!$omp section  
    <code block2>  
!$omp section  
    :  
!$omp end sections [nowait]
```

Clauses supported:

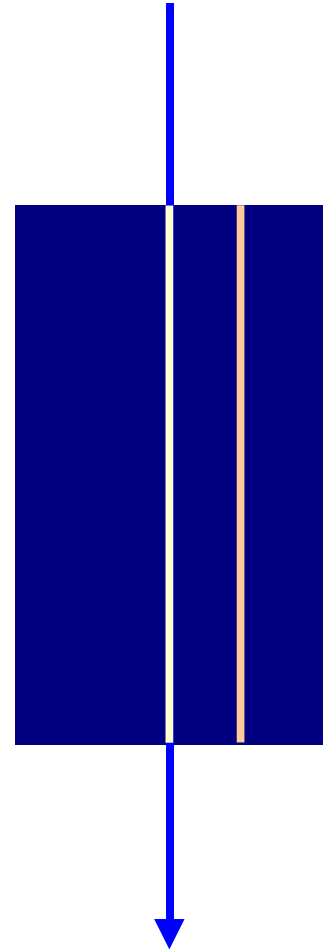
private	firstprivate
lastprivate	reduction
nowait	

Note: The SECTION directive must be within the lexical extent of the SECTIONS/END SECTIONS pair

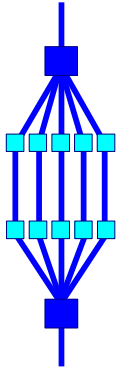
The sections directive - example



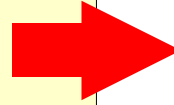
```
#pragma omp parallel default(none) \  
    shared(n,a,b,c,d) private(i)  
{  
    #pragma omp sections nowait  
    {  
        #pragma omp section  
        for (i=0; i<n-1; i++)  
            b[i] = (a[i] + a[i+1])/2;  
  
        #pragma omp section  
        for (i=0; i<n; i++)  
            d[i] = 1.0/c[i];  
  
    } /*-- End of sections --*/  
  
} /*-- End of parallel region --*/
```



Short-cuts



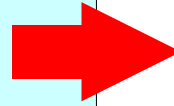
```
#pragma omp parallel
#pragma omp for
    for (...)
```



```
#pragma omp parallel for
    for (...)
```

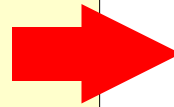
Single PARALLEL loop

```
!$omp parallel
!$omp do
    ...
!$omp end do
!$omp end parallel
```



```
!$omp parallel do
    ...
!$omp end parallel do
```

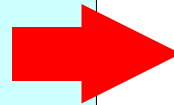
```
#pragma omp parallel
#pragma omp sections
{ ... }
```



```
#pragma omp parallel sections
{ ... }
```

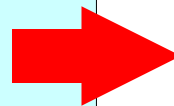
Single PARALLEL sections

```
!$omp parallel
!$omp sections
    ...
!$omp end sections
!$omp end parallel
```



```
!$omp parallel sections
    ...
!$omp end parallel sections
```

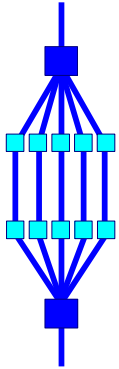
```
!$omp parallel
!$omp workshare
    ...
!$omp end workshare
!$omp end parallel
```



```
!$omp parallel workshare
    ...
!$omp end parallel workshare
```

Single WORKSHARE loop

Barrier/2



We need to have updated all of a[] first, before using a[]

```
for (i=0; i < N; i++)  
    a[i] = b[i] + c[i];
```

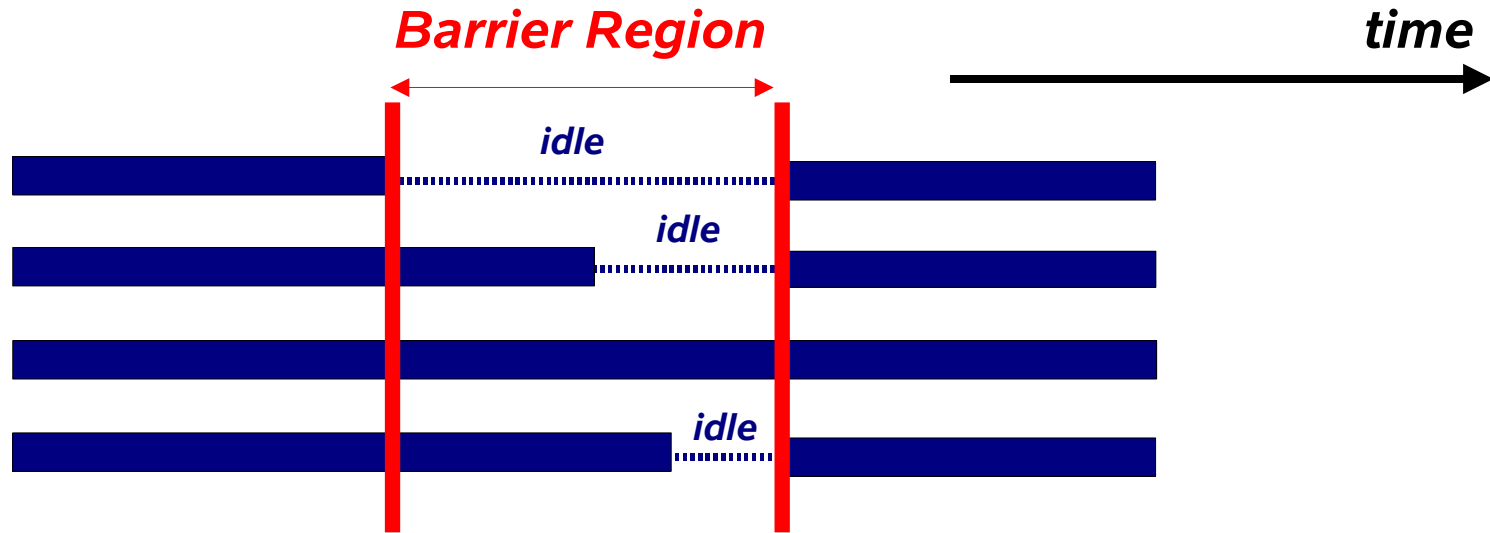
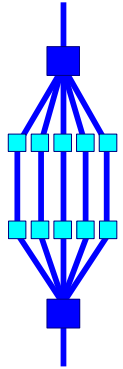
wait !

barrier

```
for (i=0; i < N; i++)  
    d[i] = a[i] + b[i];
```

All threads wait at the barrier point and only continue when all threads have reached the barrier point

Barrier/3

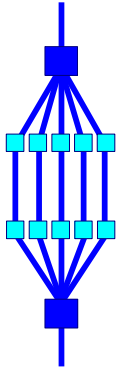


Each thread waits until all others have reached this point:

```
#pragma omp barrier
```

```
!$omp barrier
```


Critical region/1



If sum is a shared variable, this loop can not be run in parallel

```
for (i=0; i < N; i++) {  
    .....  
    sum += a[i];  
    .....  
}
```

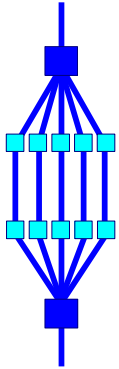
We can use a critical region for this:

```
for (i=0; i < N; i++) {  
    .....  
    sum += a[i];  
    .....  
}
```

one at a time can proceed

next in line, please

Critical region/3



All threads execute the code, but only one at a time:

```
#pragma omp critical [(name)]  
{<code-block>}
```

```
!$omp critical [(name)]  
    <code-block>  
!$omp end critical [(name)]
```

*There is no implied
barrier on entry or
exit !*

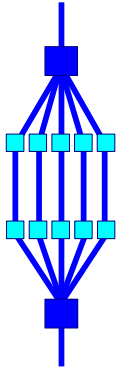
```
#pragma omp atomic  
    <statement>
```

```
!$omp atomic  
    <statement>
```

*This is a lightweight, special
form of a critical section*

```
#pragma omp atomic  
    a[indx[i]] += b[i];
```

SINGLE and MASTER construct



Only one thread in the team executes the code enclosed

```
#pragma omp single [clause[[,] clause] ...]
{
    <code-block>
}
```

```
!$omp single [clause[[,] clause] ...]
    <code-block>
!$omp end single [nowait]
```

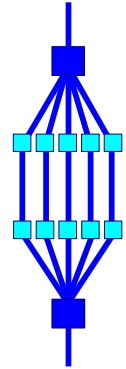
Only the master thread executes the code block:

```
#pragma omp master
{ <code-block> }
```

```
!$omp master
    <code-block>
!$omp end master
```

*There is no implied
barrier on entry or
exit !*

OpenMP environment variables

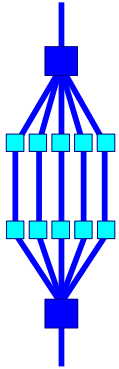


OpenMP environment variable	Default for Sun OpenMP
OMP_NUM_THREADS <u>n</u>	1
OMP_SCHEDULE “ <u>schedule</u> ,[<u>chunk</u>]”	static, “N/P” (1)
OMP_DYNAMIC { TRUE FALSE }	TRUE (2)
OMP_NESTED { TRUE FALSE }	FALSE (3)

- (1) The chunk size approximately equals the number of iterations (N) divided by the number of threads (P)*
- (2) The number of threads will be limited to the number of on-line processors in the system. This can be changed by setting **OMP_DYNAMIC** to **FALSE**.*
- (3) Multi-threaded execution of inner parallel regions in nested parallel regions is supported as of Sun Studio 10*

Note: The names are in uppercase, the values are case insensitive

Runtime library overview



Name

omp_set_num_threads
omp_get_num_threads
omp_get_max_threads
omp_get_thread_num
omp_get_num_procs
omp_in_parallel
omp_set_dynamic

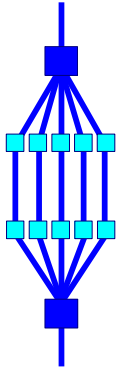
omp_get_dynamic
omp_set_nested

omp_get_nested
omp_get_wtime
omp_get_wtick

Functionality

Set number of threads
Return number of threads in team
Return maximum number of threads
Get thread ID
Return maximum number of processors
Check whether in parallel region
Activate dynamic thread adjustment
(but implementation is free to ignore this)
Check for dynamic thread adjustment
Activate nested parallelism
(but implementation is free ignore this)
Check for nested parallelism
Returns wall clock time
Number of seconds between clock ticks

Nested locking



- ❑ *Simple locks: may not be locked if already in a locked state*
- ❑ *Nestable locks: may be locked multiple times by the same thread before being unlocked*
- ❑ *In the remainder, we will discuss simple locks only*
- ❑ *The interface for functions dealing with nested locks is similar (but using nestable lock variables):*

Simple locks

```
omp_init_lock  
omp_destroy_lock  
omp_set_lock  
omp_unset_lock  
omp_test_lock
```

Nestable locks

```
omp_init_nest_lock  
omp_destroy_nest_lock  
omp_set_nest_lock  
omp_unset_nest_lock  
omp_test_nest_lock
```